01 - Foundation and App Shell

Feature objective

Create the stable mobile runtime that every later feature can depend on: Expo Router, providers, auth-aware API transport, query client, design tokens, SQLite bootstrap, offline awareness, reusable UI primitives, and navigation shell.

Do not build product screens deeply in this phase.

Dependencies

None. This is milestone zero.

In scope

- Expo app initialization
- Android-first configuration
- Expo Router
- primary tab shell
- protected app/auth/onboarding route groups
- provider composition
- TanStack Query
- Zustand baseline
- SQLite initialization
- SecureStore utility
- NetInfo integration
- API transport + SDK integration
- design tokens
- core reusable UI primitives
- global error boundary
- toast system
- basic test harness

Out of scope

- domain feature logic
- SMS parsing
- financial calculations
- production analytics
- Sentry/PostHog
- dark mode
- iOS-specific polish

Route skeleton

```
app/
|-- _layout.tsx
|-- index.tsx
|-- +not-found.tsx
|-- (auth)/
|-- (onboarding)/
`-- (app)/
    |-- _layout.tsx
    `-- (tabs)/
        |-- _layout.tsx
        |-- home.tsx
        |-- transactions.tsx
        |-- goals.tsx
```

```
        |-- plan.tsx
        `-- ai.tsx
```

Tab screens can temporarily render feature placeholders, but routes must already be real so later features attach cleanly.

Recommended provider tree

```
RootErrorBoundary
`-- GestureHandlerRootView
    `-- SafeAreaProvider
        `-- QueryClientProvider
            `-- AuthProvider
                `-- DatabaseProvider
                    `-- NetworkProvider
                        `-- NotificationProvider
                            `-- Expo Router Slot
```

Keep provider responsibilities narrow. Do not build one `AppProvider` containing the entire application.

API transport

Implement one auth-aware transport used by the shared SDK.

Responsibilities:

- configurable API base URL
- attach access token from memory
- send Bearer token for mobile
- intercept/normalize 401
- ensure only one refresh request is in flight
- read refresh token from SecureStore
- refresh session
- update in-memory access token
- retry original request once
- if refresh fails, clear auth state and redirect to login
- support AbortSignal/cancellation

Never store the access token in AsyncStorage.

SDK integration

Canonical contract:

`backend OpenAPI -> sdk/generated -> mobile feature wrappers`

Create thin feature API wrappers later rather than calling generated endpoints directly from screens.

Suggested mobile dependency:

```
{
  "dependencies": {
    "@financial-dream-planner/sdk": "file:../sdk"
  }
}
```

Validate this arrangement in EAS early. If EAS packaging becomes unreliable, publish the SDK package to a private registry rather than duplicating generated code.

Query client defaults

Recommended baseline:

- retry queries only for recoverable network/server failures
- avoid retrying auth/validation failures
- normal stale time: approximately 30-60 seconds
- cache useful dashboard/transaction data longer than stale time
- refetch on foreground for selected queries
- mutations invalidate targeted keys only

Query keys stay feature-owned.

Example:

```
['transactions', householdId, filters]
['account', accountId]
['goals', householdId]
['plan', householdId, 'current']
```

SQLite bootstrap

Create versioned migrations from day one.

Initial local tables can include:

```
local_meta
sync_state
transaction_outbox
sms_candidates
merchant_rules_cache
```

Store schema version and parser version.

Never use SQLite as a shadow copy of the entire server database without a clear reason.

Network provider

Use NetInfo to expose a small stable interface:

```
{
  isOnline: boolean,
  isInternetReachable: boolean | null
}
```

Use this for UX and outbox decisions, not as proof that an API request will succeed.

Design tokens

Create platform-native TypeScript tokens:

```
src/design/
|-- colors.ts
|-- spacing.ts
|-- radius.ts
|-- typography.ts
|-- elevation.ts
|-- motion.ts
`-- theme.ts
```

MVP has one light theme.

Core UI primitives

Build only primitives that immediately reduce duplication:

```
Button
IconButton
TextField
AmountField
Card
SectionHeader
StatusPill
Skeleton
Toast
InlineError
EmptyState
OfflineBanner
ScreenContainer
TopBar
BottomSheet wrapper
```

Do not create a 70-component design system before features exist.

Motion baseline

Use Reanimated selectively.

Recommended durations:

```
tap feedback: 100-140ms
small state transition: 160-200ms
screen/card reveal: 180-240ms
bottom sheet and gestures: spring
```

Provide a reduced-motion hook and ensure components can fall back to opacity/no-motion transitions.

Step-by-step implementation

Step 1 - Create Expo app

- TypeScript enabled
- Expo Router enabled
- Android package identifier chosen
- environment configuration separated for dev/preview/prod

Step 2 - Add route groups

- auth
- onboarding
- authenticated app
- tab shell

Step 3 - Build provider composition

- query client
- auth placeholder
- DB initialization
- network status
- toast host

Step 4 - Add SDK/API transport

- shared client construction
- token attachment
- normalized errors
- refresh skeleton

Step 5 - Add design tokens/primitives

- typography
- spacing
- colors
- Button/TextField/Card/Skeleton

Step 6 - Add local database migration system

- first migration
- migration runner
- reset helper for development only

Step 7 - Add global failure handling

- React error boundary
- not-found route
- API error normalization
- offline banner

Step 8 - Add test baseline

- Jest
- React Native Testing Library
- test renderer/provider helper

Acceptance criteria

- app launches into correct placeholder route without redbox
- tabs render and preserve navigation state
- app can render with no network connection
- QueryClient and API transport are available through feature hooks
- SQLite migration runs safely more than once
- SecureStore helper can save/read/remove a test value
- global error boundary can recover to a safe screen
- 48dp minimum touch targets for primitives
- design tokens are used instead of arbitrary repeated values

Tests

- API refresh single-flight unit test
- normalized API error tests
- database migration test
- network state hook test
- Button/TextField accessibility smoke tests
- root provider render test

Definition of Done

Foundation is done only when a later feature can be created without changing root architecture.